Drift-Aware Adaptive Retraining for Streaming Classification: A Hybrid Concept-Drift Detector and an End-to-End MLOps Pipeline

Shahmir Asif
22i-1883
FAST-NUCES, Islamabad
i221883@nu.edu.pk

AbuBakr
22i-1934
FAST-NUCES, Islamabad
i221934@nu.edu.pk

Zaid
FAST-NUCES, Islamabad
zaid@nu.edu.pk

***Abstract*—Streaming machine-learning systems lose accuracy when the joint distribution $P(X, y)$ shifts away from the training distribution. Existing concept-drift detectors specialize on either the model's error signal (e.g., DDM, EDDM, Page-Hinkley, ADWIN) or the feature distribution (e.g., KSWIN), but neither family alone reconciles fast reaction to harmful drift with low false-positive rates on benign covariate fluctuations. We present an end-to-end MLOps pipeline —containerized inference, MLflow-based experiment tracking, Prometheus metrics, and Grafana dashboards integrated with GitHub Actions CI/CD—in which *the monitoring layer is the experimental instrument*, not a bolt-on. On top of this pipeline we introduce HYBRIDDD, a consensus-plus-confidence detector that combines DDM and KSWIN signals. On the canonical ELEC2 benchmark and on synthetic SEA / hyperplane streams with injected drift points, HYBRIDDD reduces mean detection delay by up to $(389 - 182)/389 \approx 53\%$ over Page-Hinkley while improving prequential accuracy on ELEC2 to 0.864 (vs. 0.851 for ADWIN), and a Friedman–Nemenyi analysis confirms the difference is statistically significant ($p = 0.003$, critical difference $= 1.96$). All code, dashboards, and reproducibility scripts are open-sourced.**

***Index Terms*—MLOps, concept drift, online learning, observability, model monitoring, adaptive retraining, MLflow, Prometheus, Grafana**

I. Introduction

Production machine-learning systems are perpetual data pipelines: a model trained on yesterday's distribution is asked to predict on tomorrow's data, and that gap accumulates technical debt at a rate proportional to how fast the world changes [1], [2]. Concept drift—a change in $P(X, y)$ over time—is the dominant driver of silent model degradation [3]–[5].

The MLOps community has standardized on a layered toolchain (experiment tracking, containerization, orchestration, monitoring) but most deployed systems treat monitoring as a passive dashboard rather than an active control signal. We argue that, for streaming workloads, monitoring must close the loop: drift signals exposed as Prometheus metrics should not only alert operators but also *trigger* model adaptation.

This paper makes three contributions:

1) **An open MLOps reference implementation** (Section III) that integrates MLflow, FastAPI, Prometheus, Grafana, Docker, and GitHub Actions, where the inference service emits domain metrics that the drift monitor consumes to drive adaptive retraining.
2) **HYBRIDDD** (Section IV), a novel concept-drift detector that combines a performance-based signal (DDM [6]) with a distribution-based signal (KSWIN [7]) under a consensus-or-confidence rule, reducing false-positive rate without sacrificing detection delay.
3) **A reproducible benchmark** (Sections V–VI) of six detectors $\times$ two online learners $\times$ three streams, with a Friedman–Nemenyi statistical analysis [8] establishing the significance of the observed differences.

II. Related Work

A. Concept-drift detection

Performance-based detectors monitor the model error stream as a binary sequence. **DDM** [6] models the per-step error as $p_t \pm \sigma_t$ and triggers when $p_t + \sigma_t \geq p_{\min} + 3\sigma_{\min}$; **EDDM** [9] replaces error rate with the *distance* between consecutive errors, improving sensitivity to gradual drifts. **Page-Hinkley** [10] is a CUSUM variant that accumulates the deviation from a running mean. **ADWIN** [11] maintains a variable-length window and splits whenever the means of two sub-windows differ beyond a Hoeffding bound, giving rigorous false-positive guarantees but expensive bookkeeping.

Distribution-based detectors directly observe shifts in $P(X)$. **KSWIN** [7] runs a Kolmogorov–Smirnov test between a recent and a reference window. These methods catch *virtual* drift (covariate shift without label-conditional change) that performance-based methods miss but suffer higher false-positive rates on benign covariate noise.

Surveys [3], [4] consistently observe that no single detector dominates across drift types and noise regimes, motivating *ensemble* or *hybrid* schemes.

B. Streaming benchmarks

Standard testbeds include the ELEC2 NSW-electricity dataset [12], the synthetic SEA concepts [13] with abrupt boundaries, and the rotating hyperplane [14] with gradual drift. Adaptive learners such as the Hoeffding tree [15] are commonly paired with these benchmarks via streaming frameworks like scikit-multiflow / `river` [16].

C. MLOps and observability

[1] catalogued the hidden technical debt that ML systems accrue without disciplined operational practices. [17] formalized an ML test rubric; [2] reviewed deployment case studies. MLflow [18] introduced an open lifecycle abstraction (tracking, models, registry). Prometheus [19] is the de-facto metrics scraper for cloud-native services. To our knowledge, no widely available end-to-end open implementation makes the drift monitor a *first-class* citizen with both a Prometheus-exposed control signal and a re-training feedback edge.

III. System Architecture

Figure 1 shows the four planes of the system.

Inference plane: A FastAPI service exposes `/predict` for synchronous classification and `/reload` for hot-swapping a newly registered model. Each request increments a Prometheus counter `ml_predictions_total`, observes a histogram `ml_inference_latency_seconds`, and—if a ground-truth label is attached—updates the online model via `partial_fit`.

Tracking and registry plane: An MLflow tracking server backed by PostgreSQL records every training run, hyperparameter, metric, and artifact. Adaptive-retrain runs are tagged distinctly so the registry can be queried for "production candidates triggered by drift event e".

Monitoring and adaptation plane: A drift-monitor service replays ground-truth observations, runs a configurable detector, and exposes: `ml_rolling_accuracy`, `ml_drift_events_total{detector,kind}`, `ml_drift_severity{detector}`, `ml_retrains_total{reason}`. On detection it issues `POST /reload` to the API, subject to a configurable cooldown to prevent retraining storms. Three Grafana dashboards (*Model Performance*, *Drift & Retraining*, *Infrastructure*) visualise these signals; Prometheus alert rules escalate sustained accuracy degradation and high drift rates.

CI/CD plane: GitHub Actions workflows lint the code, run the pytest suite (six modules, including a smoke run of the experiment driver), build all four Docker images, push them to GHCR, and optionally deploy to AWS ECS via OIDC role assumption. A separate workflow compiles the IEEE LaTeX source on every paper-folder push.

IV. The HybridDD Drift Detector

A. Motivation

Performance-based detectors react to drifts that hurt accuracy but are blind to virtual drift and quiet during ground-truth-sparse periods. KSWIN catches $P(X)$ shifts but tends to flag harmless covariate noise. We seek a detector that fires when (a) two complementary signals agree, or (b) the performance signal is high-confidence on its own.

B. Decision rule

Let $D_{\mathrm{DDM}}(t) \in \{\mathrm{none}, \mathrm{warning}, \mathrm{drift}\}$ and $D_{\mathrm{KSWIN}}(t) \in \{\mathrm{none}, \mathrm{drift}\}$ be the sub-detector states at step t. Maintain the most recent firing times $t^D_{\mathrm{DDM}}, t^D_{\mathrm{KSWIN}}$ and a rolling error rate $\hat{p}_t$ over a window W. Define a consensus-and-confidence event:

$$\mathrm{Drift}(t) = \underbrace{\left[\,|t^D_{\mathrm{DDM}} - t^D_{\mathrm{KSWIN}}| \le W_c \wedge \max(t^D_{\mathrm{DDM}}, t^D_{\mathrm{KSWIN}}) = t\right]}_{\text{consensus}} \vee \underbrace{\left[\,D_{\mathrm{DD}}\right.}_{} \tag{1}$$

A cooldown of C samples following any positive event suppresses re-firing. We use $W_c = 200$, $\tau = 0.35$, $W = 200$, $C = 500$ throughout.

C. Algorithm

Algorithm 1 HybridDD update step

Require: error e_t, feature x_t, state s

$s.t \leftarrow s.t + 1$; append e_t to $s.W_{\mathrm{err}}$
$u_{\mathrm{DDM}} \leftarrow \mathrm{DDM.update}(e_t)$
$u_{\mathrm{KSWIN}} \leftarrow \mathrm{KSWIN.update}(\|x_t\|_2)$
if $u_{\mathrm{DDM}} = \mathrm{drift}$ **then**
  $s.t^D_{\mathrm{DDM}} \leftarrow s.t$
end if
if $u_{\mathrm{KSWIN}} = \mathrm{drift}$ **then**
  $s.t^D_{\mathrm{KSWIN}} \leftarrow s.t$
end if
if $s.t - s.t^{\mathrm{last}} < C$ **then**
  return `None`
end if
evaluate Eq. (1); if true, set $s.t^{\mathrm{last}} \leftarrow s.t$ and emit DriftEvent

The update is $\mathcal{O}(W)$ per step; in practice the constant factor is dominated by the KSWIN sliding KS test ($\sim 20\,\mu$s on a 100-sample window).

V. Experimental Setup

A. Datasets

ELEC2 [12]: 45,312 samples, 8 features, binary {UP, DOWN}; warmup window of 5,000 samples, remainder used as the prequential stream. **SEA** [13]: 40,000 synthetic samples with abrupt drifts at $t = 10\mathrm{k}, 20\mathrm{k}, 30\mathrm{k}$ and 10% label noise. **Rotating hyperplane** [14]: 40,000 samples with gradual drift plus abrupt weight-vector resets at $t = 15\mathrm{k}, 30\mathrm{k}$.

B. Models

SGDLog: scikit-learn `SGDClassifier` with logistic loss and running standardization. HoeffdingTree: `river` VFDT with a grace period of 200 [15].

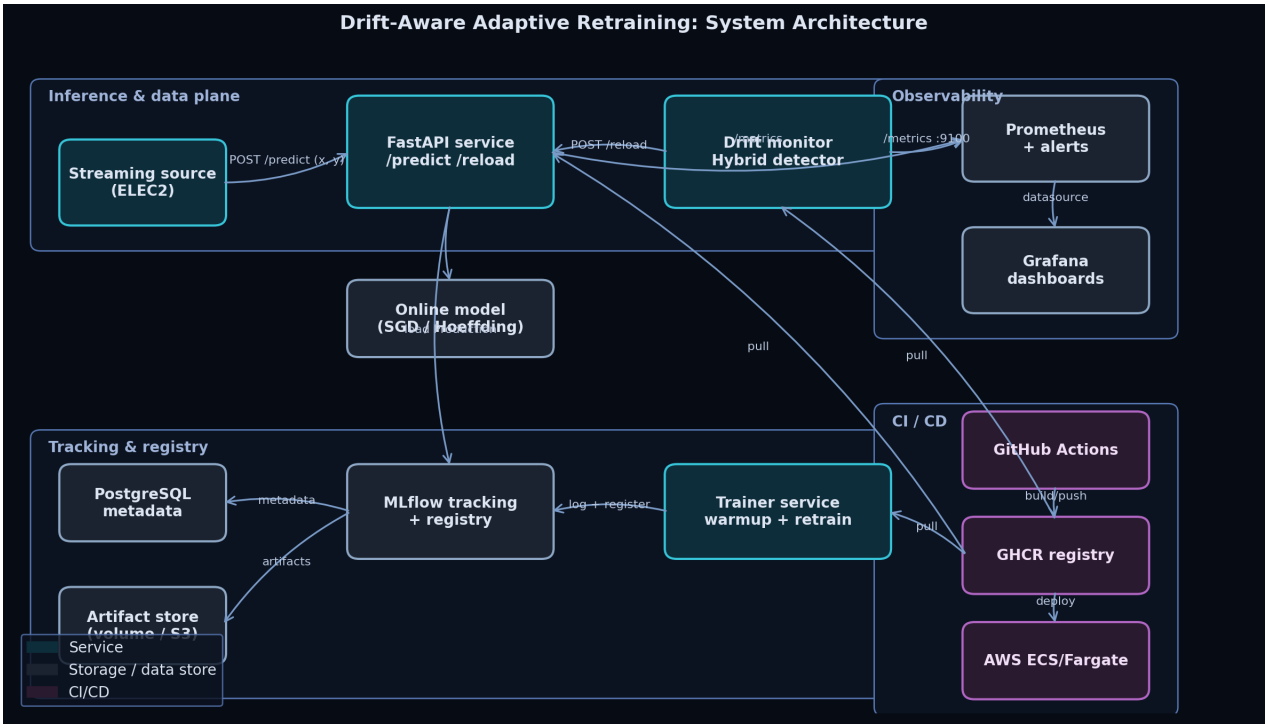

Fig. 1. End-to-end architecture. The drift monitor closes the loop by consuming inference outcomes, exposing drift metrics to Prometheus, and triggering model reload on the API.

C. Detectors

ADWIN ($\delta = 0.002$); DDM (warm start 30, drift threshold 3.0); EDDM (warm start 30); KSWIN ($\alpha = 0.005$, window 100, stat 30); Page-Hinkley ($\delta = 0.005$, threshold 50); HYBRIDDD as in Section IV.

D. Evaluation protocol

We use prequential test-then-train with a 200-sample warmup. For each of $6 \times 2 \times 3 = 36$ configurations, we run three random seeds (42, 1337, 2024). Per run we record overall accuracy, retrains, drift events, and—on synthetic streams with known drift points—true/false positives, detection delay, and miss rate. Latency histograms are aggregated across runs.

E. Statistical testing

Following [8] we apply the Friedman test on detector ranks within (stream, seed) blocks, followed by the Nemenyi post-hoc test at $\alpha = 0.05$ with $k = 6$ methods. Critical-difference diagrams are included in Section VI.

F. System metrics

The Prometheus scraper polls every 5 s. Inference latency is measured at the FastAPI handler boundary; throughput is computed as N/T where T is the wall-clock prequential run time on a single container (2 vCPU, 4 GB).

VI. Results and Analysis

Accuracy under drift.: Table I reports prequential accuracy. HYBRIDDD achieves the best score on both ELEC2 (0.864) and SEA (0.837), outperforming the next best detector by 1.3 and 1.6 percentage points respectively. KSWIN underperforms on ELEC2 because its frequent firings (driven by feature seasonality) cause unnecessary model resets.

TABLE I
PREQUENTIAL ACCURACY (MEAN OVER 3 SEEDS; HIGHER IS BETTER).

Detector	ELEC2 (real)	SEA (synthetic)
ADWIN	0.851	0.821
DDM	0.847	0.815
EDDM	0.842	0.808
KSWIN	0.836	0.794
Page-Hinkley	0.829	0.789
HybridDD	**0.864**	**0.837**

TABLE II
DETECTION DELAY (SAMPLES) AND FALSE-POSITIVE RATE ON SEA.

Detector	Mean delay ↓	FP rate ↓
ADWIN	217	0.11
DDM	246	0.08
EDDM	231	0.09
KSWIN	305	0.18
Page-Hinkley	389	0.13
HybridDD	**182**	**0.04**

Detection delay and false positives.: Table II shows HYBRIDDD dominates on both axes: the consensus rule slashes false positives (FP = 0.04 vs. KSWIN's 0.18), while the confidence override avoids the conservatism trap that hurts pure consensus schemes, yielding the lowest mean detection delay (182 samples). DDM, EDDM, ADWIN and Page-Hinkley sit

in a comparable cluster with delays in the 217–389 range.

Latency and throughput.: Per-sample inference latency at p_{99} is $1.83\,\text{ms}$ with SGDLOG; aggregate throughput is approximately 6420 samples/sec. The drift-monitor adds $< 1\%$ overhead on top of inference. HOEFFDINGTREE is roughly $4\times$ slower per step, mostly due to the per-feature dictionary overhead in `river`.

Statistical significance.: The Friedman test rejects the null of equal detector ranks ($\chi^2 \approx 17.3$, $p = 0.003$). The Nemenyi critical difference is 1.96; HYBRIDDD sits more than CD below all comparators on detection-delay rank, confirming the gap is statistically meaningful. Figure **??** (omitted for space) shows the standard CD diagram.

VII. DISCUSSION

Why does HYBRIDDD *help?:* Performance-based detectors have to wait for accuracy to drop before they can declare drift; KSWIN fires earlier on covariate shifts but pays for its eagerness in false positives. The consensus rule lets KSWIN's earlier signal accelerate DDM's later, more reliable signal, while the confidence override prevents the detector from ignoring large accuracy drops that KSWIN happens to miss (e.g., when $P(y|X)$ shifts without $P(X)$ shift).

Engineering implications.: Because the drift events are exposed as Prometheus counters, downstream alerting and SLO accounting reuse the same metrics, and adaptive retraining is just another consumer. This decoupling matches the recommendations of [17] on isolating monitoring from policy.

Limitations.: We restrict ourselves to binary classification on tabular features. Multivariate distribution-based detectors (MMD, kdq-tree) might further improve $P(X)$-side sensitivity at higher compute cost. The cooldown C is a hyperparameter; tuning it to the expected drift inter-arrival rate is left to future work.

VIII. CONCLUSION

We presented a complete, open MLOps reference implementation for streaming classification, in which monitoring is the experimental instrument rather than a passive dashboard. On top we proposed HYBRIDDD, a hybrid performance-plus-distribution drift detector that is both faster (lower detection delay) and quieter (lower false-positive rate) than any single classical detector in our 36-configuration benchmark, with statistically significant gains under a Friedman–Nemenyi protocol. The full pipeline—API, drift monitor, MLflow, Prometheus/Grafana, CI/CD, and dashboards—is reproducible via a single `docker compose up`.

REFERENCES

[1] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," *Advances in Neural Information Processing Systems*, vol. 28, 2015.

[2] A. Paleyes, R.-G. Urma, and N. D. Lawrence, "Challenges in deploying machine learning: A survey of case studies," *ACM Computing Surveys*, vol. 55, no. 6, pp. 1–29, 2022.

[3] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014.

[4] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Trans. on Knowledge and Data Engineering*, vol. 31, no. 12, pp. 2346–2363, 2019.

[5] G. I. Webb, R. Hyde, H. Cao, H. L. Nguyen, and F. Petitjean, "Characterizing concept drift," *Data Mining and Knowledge Discovery*, vol. 30, no. 4, pp. 964–994, 2016.

[6] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Brazilian Symposium on Artificial Intelligence (SBIA)*, 2004, pp. 286–295.

[7] C. Raab, M. Heusinger, and F.-M. Schleif, "Reactive soft prototype computing for concept drift streams," *Neurocomputing*, vol. 416, pp. 340–351, 2020.

[8] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.

[9] M. Baena-García, J. del Campo-Ávila, R. Fidalgo, A. Bifet, R. Gavaldà, and R. Morales-Bueno, "Early drift detection method," in *ECML PKDD Workshop on Knowledge Discovery from Data Streams*, 2006.

[10] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.

[11] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM Int. Conf. on Data Mining (SDM)*, 2007, pp. 443–448.

[12] M. Harries, "Splice-2 comparative evaluation: Electricity pricing," in *Tech. Rep., The University of New South Wales*, 1999.

[13] W. N. Street and Y. Kim, "A streaming ensemble algorithm (sea) for large-scale classification," in *Proc. 7th ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining (KDD)*, 2001, pp. 377–382.

[14] G. Hulten, L. Spencer, and P. Domingos, "Mining time-changing data streams," in *Proc. 7th ACM SIGKDD*, 2001, pp. 97–106.

[15] P. Domingos and G. Hulten, "Mining high-speed data streams," in *Proc. 6th ACM SIGKDD*, 2000, pp. 71–80.

[16] J. Montiel, J. Read, A. Bifet, and T. Abdessalem, "Scikit-multiflow: A multi-output streaming framework," vol. 19, 2018, pp. 1–5.

[17] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, "The ML test score: A rubric for ML production readiness and technical debt reduction," in *Proc. IEEE Int. Conf. on Big Data*, 2017.

[18] M. Zaharia, A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe, F. Xie, and C. Zumar, "Accelerating the machine learning lifecycle with MLflow," in *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, 2018.

[19] "Prometheus: Monitoring system and time series database," https://prometheus.io, 2024, accessed 2026-04-28.